Inset 1: Order of symbol resolution

In the *Makefile*, you must make sure that the created object files are placed before the *pkg-config* generated link flags. For example, if the *Makefile* contains the following code:

```
employee: employee.o
	$(CC) $< $(LFLAGS) -o $@
```

This effectively expands to what's shown below:

```
gcc employee.o <value of LFLAGS> -o employee
```

So, in Windows, GCC will try to locate symbols used in *employee*.o using the *LFLAGS* value. If you change the order as indicated below…

```
employee: employee.o
	$(CC) $(LFLAGS) -o $@ $<
```

…GCC will NOT use the value of *LFLAGS* to resolve symbols for *employee.o*, because it (*employee.o*) appeared after LFLAGS. You will get a large number of *symbol not found* errors, and the application will fail to link.

```
C:\msys\1.0 on / type user (binmode,noumount)
C:\msys\1.0 on /usr type user (binmode,noumount)
c:\mingw on /mingw type user (binmode)
c: on /c type user (binmode,noumount)
d: on /d type user (binmode,noumount)
l: on /l type user (textmode)
om@Desktop::~$] mount L:/ /mnt/l/
om@Desktop::~$] cp -r /mnt/l/prg.git/employee/employee-code .
```

Compiling

There are some important points to note while porting applications to Windows. Depending on the complexity of the project, the changes required may range from *Makefile*s to C source files. First, since normal executable file names in Windows end with the *.exe* extension, make sure that you change the target name in the *Makefile* to include the *.exe* suffix. Otherwise, if, for instance, the target name is *employee*, every time you run the *make* program, it won't find the target, because it produced *employee.exe* when it ran the last time—and will try to re-run the commands to create this target. Second, make sure that all the changes you make in the source code are conditionally compiled. You can wrap such code within Windows-specific *#ifdef ... #endif* instructions, like *#ifdef _WIN32* , *#ifdef WINDOWS*, etc.

If you are working with path names, make sure you have a way to switch between Windows path separators (\ - backslash) and UNIX path separators (/ - forward slash). Also, do remember that in C, a literal \ is an escape sequence in a string; you need to use \\ to represent \ in the path string(s).

I have noticed that the order of symbol resolution is stricter in Windows than Linux. Please read Inset 1 carefully; that could save you hours of frustration later.

Debugging with GDB

You need the GDB package installed. Download it from *http://sourceforge.net/downloads/mingw/MinGW/BaseSystem/GDB/GDB-7.1/gdb-7.1-2-mingw32-bin.tar.gz/* to your home directory, and issue the following command:

```
$ cd /
$ tar xvzf ~/gdb-7.1-2-mingw32-bin.tar.gz
```

Figure 1 gives an example of debugging, at a breakpoint where *employee.exe* is stopped by *gdb*.

Message window

It is useful to have a debug window during the development stage of an application, so that you can effectively use simple *printf()* and other functions with *stdin*, *stdout*, *stderr* . Figure 2 shows the result of double clicking the employee.exe application when compiled to create a debug console. Note the background console window that acts as *stdout* and *stderr*. This does not require any special option while compiling. To have no console window (like almost all applications in MS Windows), after the application is fully debugged, include the flag *-mwindows* in the *Makefile* as a flag to *gcc*.

Packaging and distribution

The resulting application can be packaged and distributed using many available Windows packagers/installers (like Microsoft Installer, for example). You don't need all the programs we installed while setting up the development environment; only the libraries your application requires.

Running the application

After you are done debugging, just execute the program, either from the command line or by double-clicking it in Windows Explorer.

In this four-part tutorial series, we started with the basics of how to use Glade to create a GUI front end, using Perl to add glue code. We then moved on to use C and GLib to create Linux executables, and finally explored how to port C code to Windows to create Windows executables. I enjoyed writing this series, and strove to keep a balance between being technically informative, yet easy to follow. I hope you find that it has met those objectives. END

By: Om Narasimhan

Om Narasimhan is a Linux Kernel engineer at NetLogic Microsystems, Santa Clara, California. He occasionally dabbles with GTK, Perl and Bash.